



Módulo 3

Fundamentos de animación 2D

Preparado por: Ing. Samuel Jiménez



Unidad 2

Conceptos fundamentales de la herramienta

Objetivos Generales

- ***Conocer las herramientas que permiten el dibujo de primitivas básicas.***
- ***Presentar las funciones de OpenGL que permiten el dibujo de líneas aplicando los patrones de líneas.***
- ***Mostrar las funcionalidades de OpenGL que permiten el dibujo de polígonos, rectángulos y relleno de polígonos.***

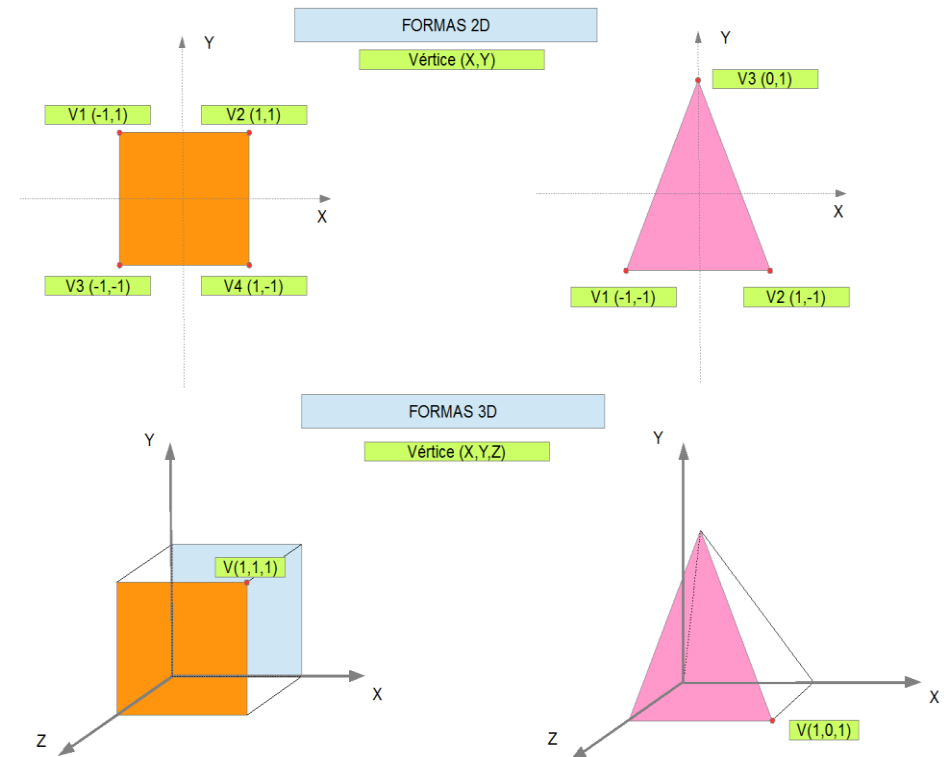


Primitivas geométricas básicas

• Función glVertex

Define las coordenadas 3D de los vértices de los puntos, líneas y polígonos especificados.

La función glVertex no puede ser llamado fuera del ámbito de un par de glBegin–glEnd, que son utilizados para delimitar los vértices de una primitiva.



Primitivas geométricas básicas

- ***Función GL_POINTS***

GL_POINTS

Puntos individuales aislados

```
glBegin(GL_POINTS);  
glVertex3f(0, 0, 0);  
glVertex3f(0, 1, 0);  
glVertex3f(1, 1, 0);  
glVertex3f(1, 0, 0);  
glEnd();
```



(0, 1, 0) (1, 1, 0)
(0, 0, 0) (1, 0, 0)

Primitivas geométricas básicas

- ***Función GL_LINES***

GL_LINES

Cada par de puntos corresponde a una recta

```
glBegin(GL_LINES);  
glVertex3f(0, 0, 0);  
glVertex3f(0, 1, 0);  
glVertex3f(1, 1, 0);  
glVertex3f(1, 0, 0);  
glEnd();
```



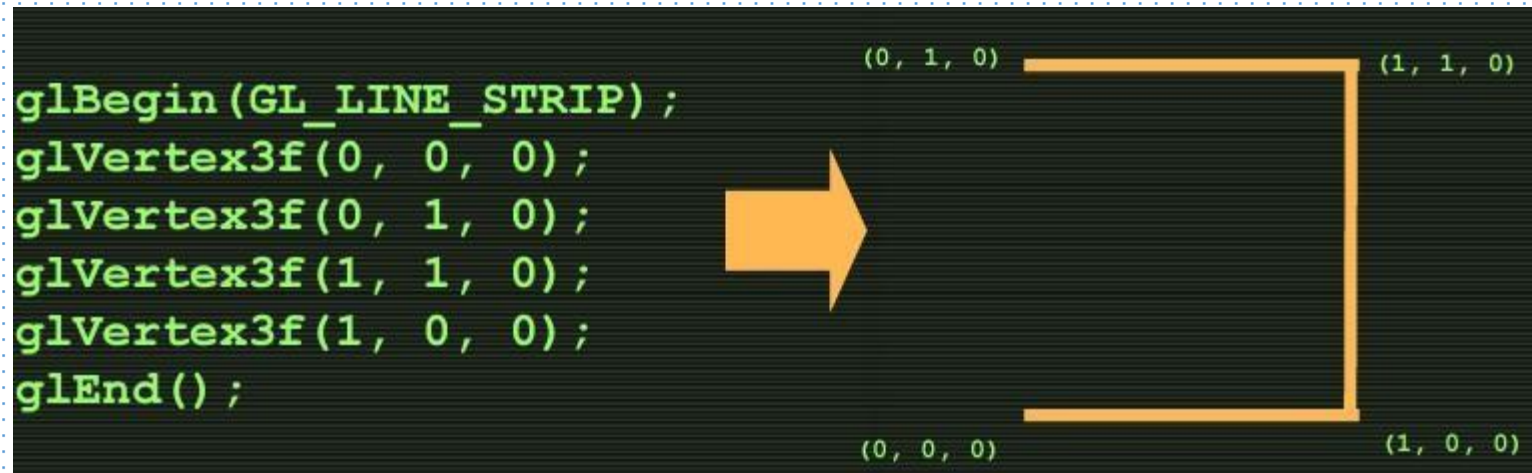
(0, 1, 0) ————— (1, 1, 0)

(0, 0, 0) ————— (1, 0, 0)

Primitivas geométricas básicas

- ***Función `GL_LINE_STRIP`***

`GL_LINE_STRIP` Segmentos de recta conectados



Conecta cada vértice con el siguiente en orden, formando una cadena abierta.

Primitivas geométricas básicas

- ***Función GL_LINE_LOOP***

GL_LINE_LOOP

Segmentos de recta conectados y cerrados

```
glBegin(GL_LINE_LOOP);  
glVertex3f(0, 0, 0);  
glVertex3f(0, 1, 0);  
glVertex3f(1, 1, 0);  
glVertex3f(1, 0, 0);  
glEnd();
```



Igual que GL_LINE_STRIP, pero conecta el último vértice con el primero (cierra el polígono).

Primitivas geométricas básicas

- ***Función GL_TRIANGLES***

GL_TRIANGLES Cada tres puntos un triángulo

```
glBegin(GL_TRIANGLES) ;  
glVertex3f(0, 1, 0);  
glVertex3f(1, 1, 0);  
glVertex3f(1, 0, 0);  
glEnd();
```



Cada 3 vértices forman un triángulo independiente.

Primitivas geométricas básicas

- ***Función GL_QUADS***

GL_QUADS

Cada cuatro puntos un cuadrado

```
glBegin(GL_QUADS);  
glVertex3f(0, 0, 0);  
glVertex3f(0, 1, 0);  
glVertex3f(1, 1, 0);  
glVertex3f(1, 0, 0);  
glEnd();
```



(0, 1, 0)

(1, 1, 0)

(0, 0, 0)

(1, 0, 0)

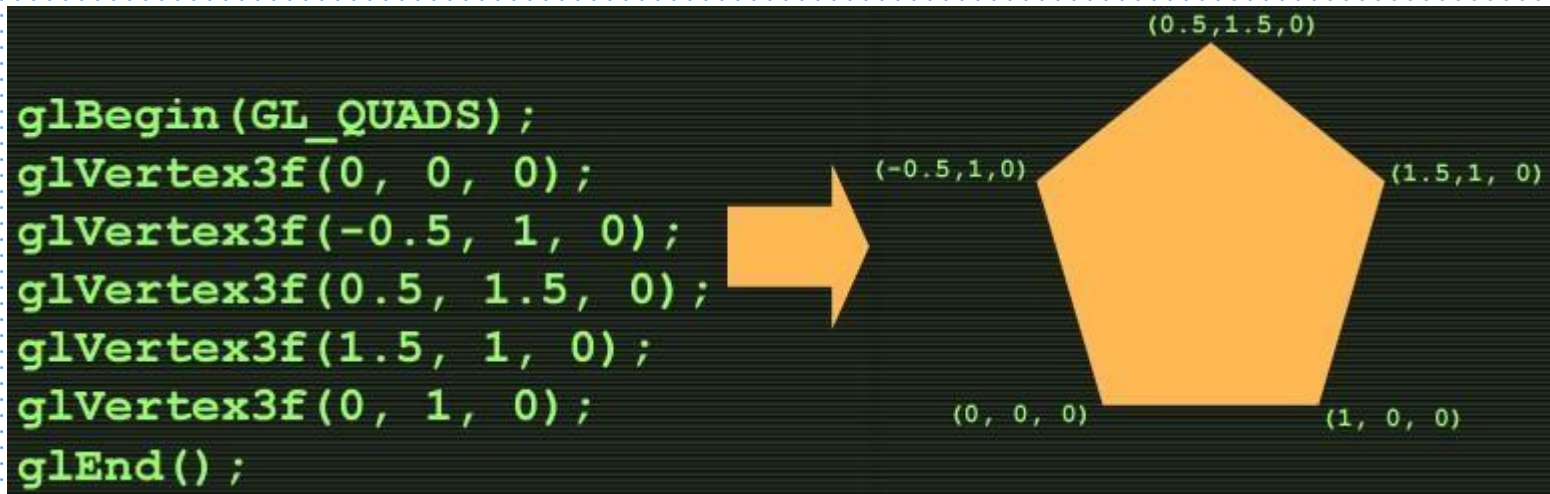


Cada grupo de 4 vértices forma un cuadrilátero.

Primitivas geométricas básicas

- ***Función GL_POLYGON***

GL_POLYGON	Un polígono
------------	-------------

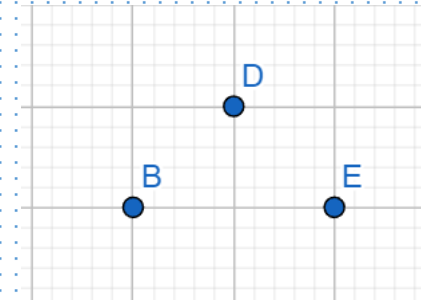


Conecta varios vértices en un polígono relleno.

Primitivas geométricas básicas

• ***Función GL_POINTS***

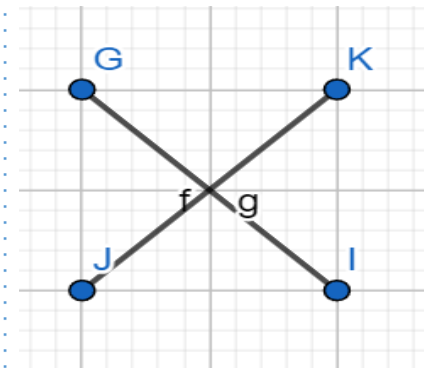
```
// ===== GL_POINTS =====  
glColor3f(1.0, 0.0, 0.0); // Rojo  
glBegin(GL_POINTS);  
    glVertex2f(-0.9f, 0.8f);  
    glVertex2f(-0.8f, 0.9f);  
    glVertex2f(-0.7f, 0.8f);  
glEnd();
```



Primitivas geométricas básicas

- ***Función GL_LINES***

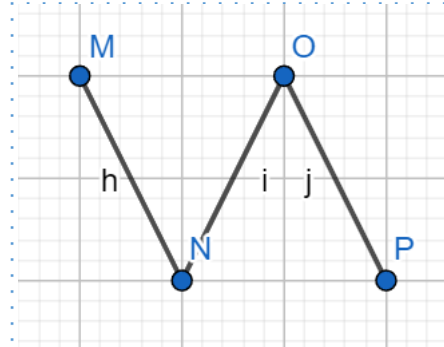
```
// ===== GL_LINES =====  
glColor3f(0.0, 0.0, 1.0); // Azul  
glBegin(GL_LINES);  
    glVertex2f(-0.5f, 0.9f);  
    glVertex2f(-0.3f, 0.7f);  
    glVertex2f(-0.5f, 0.7f);  
    glVertex2f(-0.3f, 0.9f);  
glEnd();
```



Primitivas geométricas básicas

• Función GL_LINE_STRIP

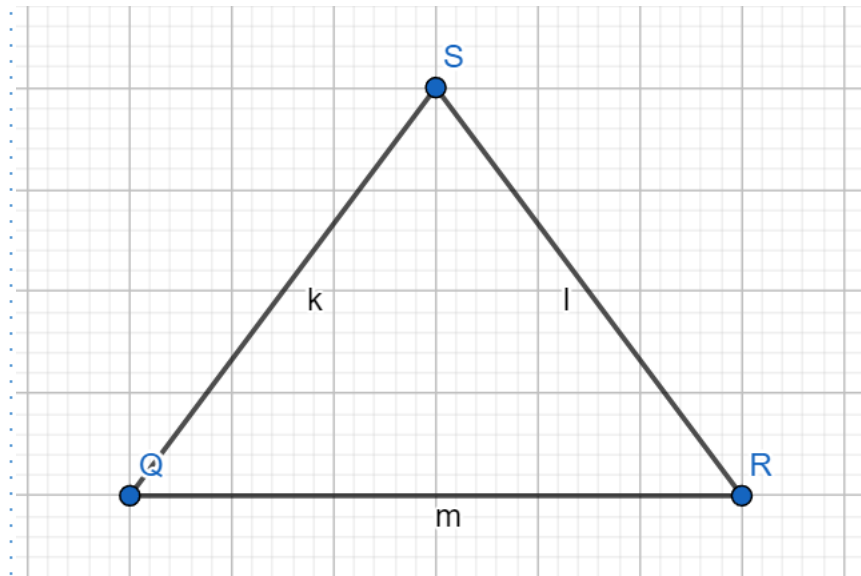
```
// ===== GL_LINE_STRIP =====  
glColor3f(0.0, 0.6, 0.0); // Verde  
glBegin(GL_LINE_STRIP);  
    glVertex2f(0.1f, 0.9f);  
    glVertex2f(0.2f, 0.7f);  
    glVertex2f(0.3f, 0.9f);  
    glVertex2f(0.4f, 0.7f);  
glEnd();
```



Primitivas geométricas básicas

• Función `GL_LINE_LOOP`

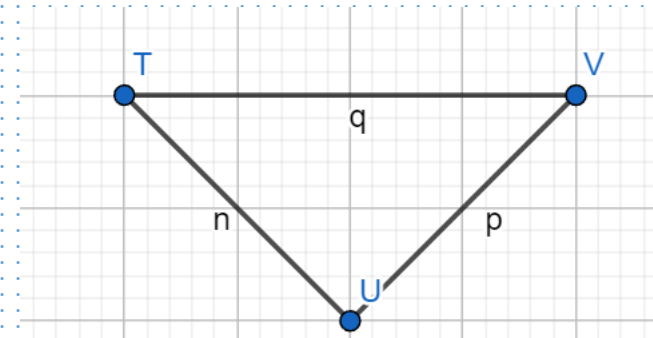
```
// ===== GL_LINE_LOOP =====  
glColor3f(0.8, 0.5, 0.0); // Naranja  
glBegin(GL_LINE_LOOP);  
    glVertex2f(0.6f, 0.7f);  
    glVertex2f(0.9f, 0.7f);  
    glVertex2f(0.75f, 0.9f);  
glEnd();
```



Primitivas geométricas básicas

• ***Función GL_TRIANGLES***

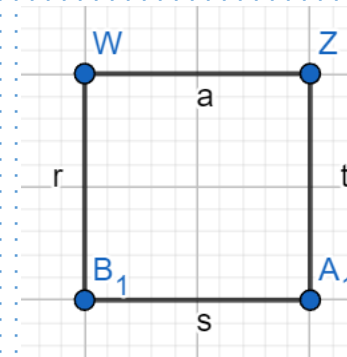
```
// ===== GL_TRIANGLES =====  
glColor3f(0.6, 0.0, 0.6); // Morado  
glBegin(GL_TRIANGLES);  
    glVertex2f(-0.9f, 0.3f);  
    glVertex2f(-0.7f, 0.1f);  
    glVertex2f(-0.5f, 0.3f);  
glEnd();
```



Primitivas geométricas básicas

• ***Función GL_QUADS***

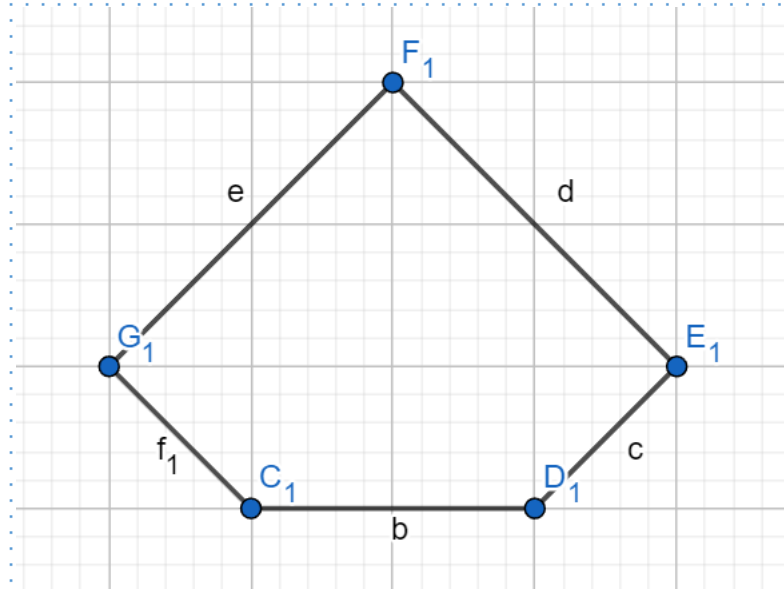
```
// ===== GL_QUADS =====  
glColor3f(0.0, 0.8, 0.8); // Turquoise  
glBegin(GL_QUADS);  
    glVertex2f(-0.3f, 0.3f);  
    glVertex2f(-0.1f, 0.3f);  
    glVertex2f(-0.1f, 0.1f);  
    glVertex2f(-0.3f, 0.1f);  
glEnd();
```



Primitivas geométricas básicas

• Función `GL_POLYGON`

```
// ===== GL_POLYGON =====  
glColor3f(0.5, 0.5, 0.5); // Gris  
glBegin(GL_POLYGON);  
    glVertex2f(0.2f, 0.1f);  
    glVertex2f(0.4f, 0.1f);  
    glVertex2f(0.5f, 0.2f);  
    glVertex2f(0.3f, 0.4f);  
    glVertex2f(0.1f, 0.2f);  
glEnd();
```



Primitivas geométricas básicas

- Ejemplo de glVertex

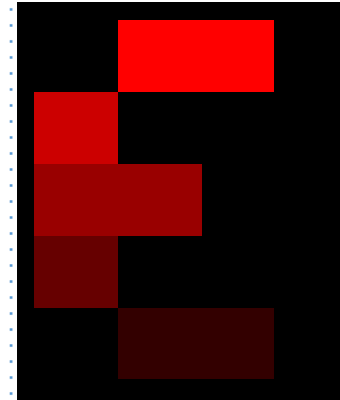
```
glColor3f(0.2,0.0,0);  
glVertex2d(-22.8,0);  
glVertex2d(-21.8,0);
```

```
glColor3f(0.4,0.0,0);  
glVertex2d(-24,2);
```

```
glColor3f(0.6,0.0,0);  
glVertex2d(-24,4);  
glVertex2d(-22.8,4);
```

```
glColor3f(0.8,0.0,0);  
glVertex2d(-24,6);
```

```
glColor3f(1,0,0);  
glVertex2d(-22.8,8);  
glVertex2d(-21.8,8);
```



Primitivas geométricas básicas

- Ejemplo de glVertex

```
glColor3f(0.2157,0.0431,0.2392);
```

```
glVertex2d(-9,0);
```

```
glVertex2d(-6.6,0);
```

```
glColor3f(0.4353,0.0863,0.4784);
```

```
glVertex2d(-9,2);
```

```
glVertex2d(-6.6,2);
```

```
glColor3f(0.6588,0.1333,0.7294);
```

```
glVertex2d(-9,4);
```

```
glVertex2d(-7.8,4);
```

```
glColor3f(0.7961,0.1569,0.8784);
```

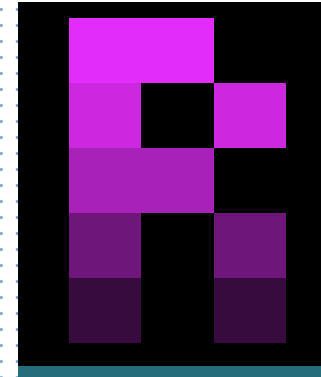
```
glVertex2d(-9,6);
```

```
glVertex2d(-6.6,6);
```

```
glColor3f(0.8863,0.1765,0.9804);
```

```
glVertex2d(-9,8);
```

```
glVertex2d(-7.8,8);
```

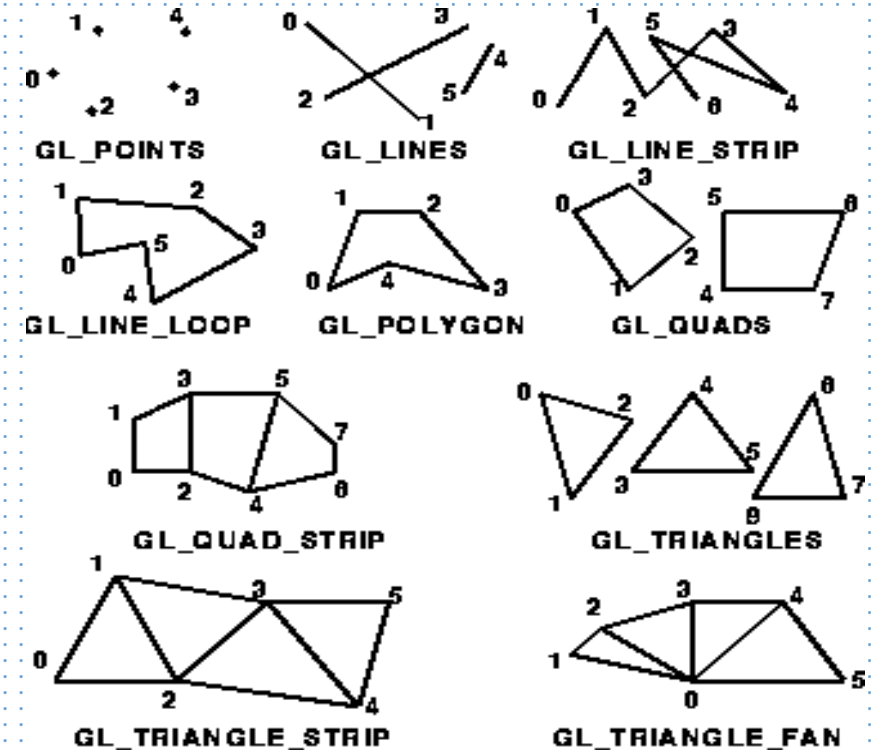


Primitivas geométricas básicas

• Función *glBegin*

Denota el inicio de un grupo de vértices que definen uno o mas primitivas.

Delimita el comienzo de especificación de vértices primitivos.



Primitivas geométricas básicas

• Función glBegin

Dentro de una secuencia glBegin–glEnd solo se puede hacer llamado a estas funciones:

- ✓ ***glVertex***
- ✓ ***glColor***
- ✓ ***glNormal***
- ✓ ***glEvalCoord***
- ✓ ***glCallList***
- ✓ ***glTextCoord***
- ✓ ***glEdgeFlag***
- ✓ ***glMaterial***

Primitivas geométricas básicas

- ***Función glBegin***

Void glBegin (GLenum mode)

La siguiente tabla muestra los valores posibles admitidos por el parámetro mode de la función glBegin que se unen formando estructuras, como líneas y polígonos.

Primitivas geométricas básicas

• ***Función glBegin***

Parámetros	Definición
GL_LINE_LOOP	Crea segmentos de rectas conectadas y cerrados
GL_LINE_STRIP	Crea segmentos de recta conectados
GL_LINES	Crea segmentos de línea, cada par de vértices corresponde a un solo segmento de línea independiente
GL_POINTS	Crea puntos individuales aislados

GL_POLYGON	Se usa para crear un polígono convexo
GL_QUAD_STRIP	Se usa para crear una banda de cuadrados
GL_QUADS	<u>Cada cuatro vértices se usa para crear un cuadrado</u>
GL_TRIANGLE_FAN	Se usa para crear un abanico de triángulos
GL_TRIANGLE_STRIP	Se usa para crear una banda de triángulos
GL_TRIANGLES	Se usa para crear triángulos, <u>cada tres vértices se especifica un nuevo triángulo</u>

Primitivas geométricas básicas

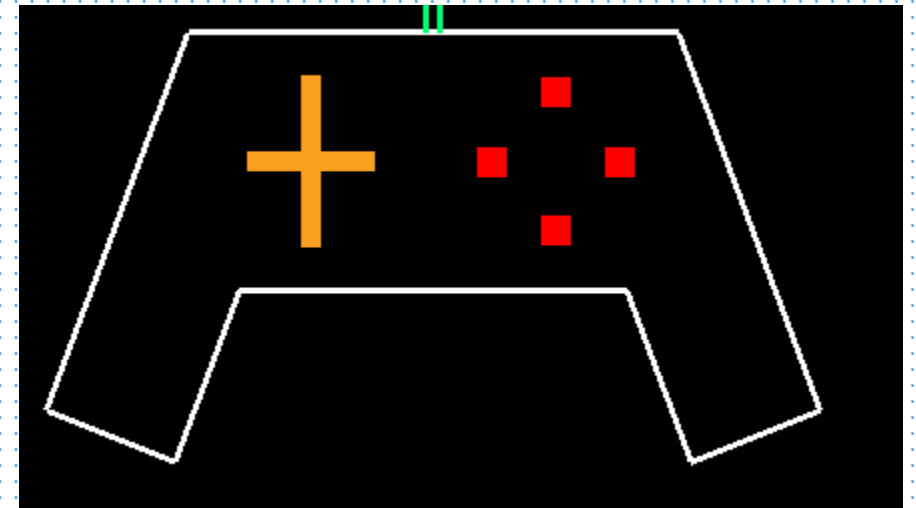
- ***Función glEnd***

Delimita el final de especificación de vértices de primitivas OpenGL.

Dentro de un solo par de glBegin – glEnd se puede incluir múltiples conjuntos de vértices, siempre que sean para el mismo tipo de primitivo

Ejemplo

```
glLineWidth(3.0);  
glBegin (GL_LINES);  
glColor3f(1.0,1.0,1.0);  
glVertex2f(-3,-5.5);glVertex2f(3,-5.5);  
glVertex2f(3,-5.5); glVertex2f(4,-6.5);  
glVertex2f(4,-6.5); glVertex2f(6,-6.2);  
glVertex2f(6,-6.2); glVertex2f(3.8,-4);  
glVertex2f(3.8,-4);glVertex2f(-3.8,-4);  
glVertex2f(-3.8,-4); glVertex2f(-6,-6.2);  
glVertex2f(-6,-6.2);glVertex2f(-4,-6.5);  
glVertex2f(-4,-6.5);glVertex2f(-3,-5.5);  
glEnd();
```



Primitivas geométricas básicas

- ***Función glFlush***

Suele utilizarse al final de una secuencia de ordenes de dibujo para asegurar que todas las primitivas gráficas de la escena son dibujadas.

Primitivas geométricas básicas

- ***Función `glRect`***

Dibuja un rectángulo plano.

Dibuja el rectángulo definido por los vértices (x_1, y_1) y (x_2, y_2) .

El rectángulo que se genera se dibuja en el plano $z=0$ y sus lados son paralelos a los ejes x e y .

(x_1, y_1) especifica la esquina superior izquierda del rectángulo y (x_2, y_2) especifica la esquina inferior derecha del rectángulo.

Primitivas geométricas básicas

```
#include <windows.h>
#include <GL/glut.h>

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // Fondo blanco
    glClearColor(1.0, 1.0, 1.0, 1.0);

    // Color azul
    glColor3f(0.0, 0.0, 1.0);

    // Dibujar un rectángulo con glRectf
    // Se indican dos esquinas opuestas (x1, y1) y (x2, y2)
    glRectf(-0.5f, -0.5f, 0.5f, 0.5f);

    glFlush();
}
```



Primitivas geométricas básicas

Grosor de punto y Línea

En OpenGL un punto, por defecto, se dibuja como un píxel en la pantalla, y las líneas se dibujan como segmentos continuos de un grosor de un píxel.

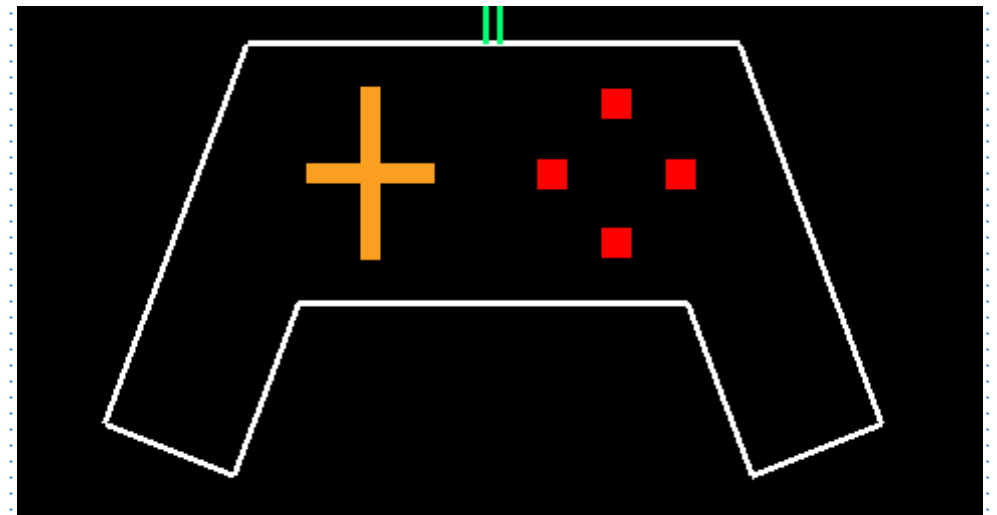
Función `glPointSize`

Se especifica el tamaño de los puntos, los tamaños varían de 0.5 a 10.0 con una granulosidad de 0.125

`Void glPointSize(GlfloatSize)`

Ejemplo

```
glPointSize(15.0);  
glBegin (GL_POINTS);  
glColor3f(1.0,0.0,0.0);  
glVertex2f(1.9,-5.15);  
glVertex2f(1.9,-4.35);  
glVertex2f(2.9,-4.75);  
glVertex2f(0.9,-4.75);  
glEnd();
```



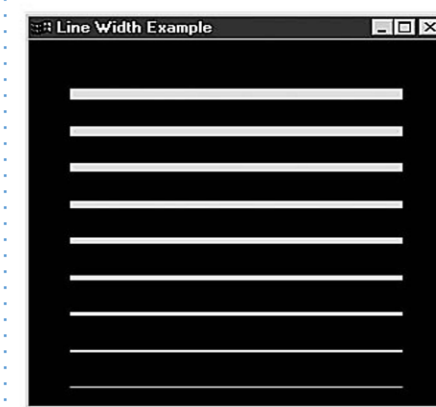
Primitivas geométricas básicas

Función `glLineWidth`

Especifica el ancho de las líneas dibujadas.

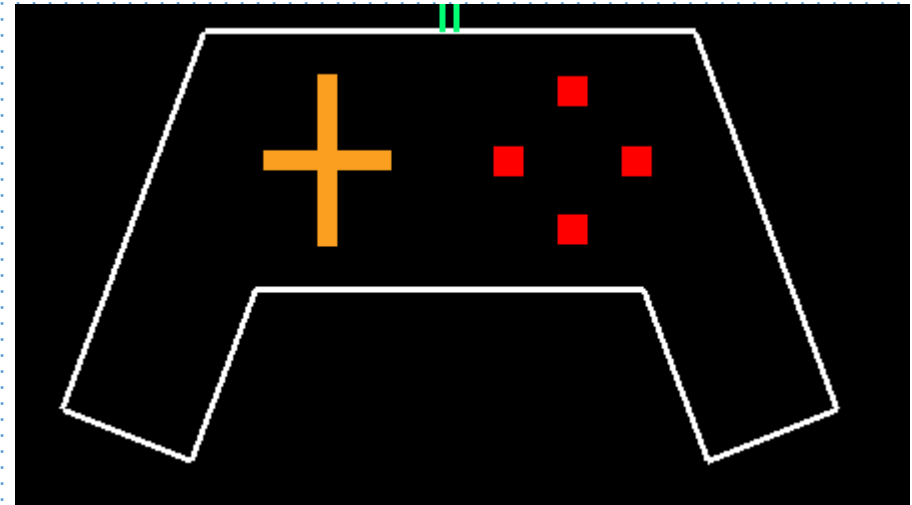
`Void glLineWidth (GLfloat width)`

Los anchos de líneas varían de 0.5 a 10.0 con una granulosidad de 0.125



Ejemplo

```
glLineWidth(10.0);  
glBegin (GL_LINES);  
glColor3f(0.98,0.625,0.12);  
glVertex2f(-1.9,-5.25);glVertex2f(-1.9,-4.25);  
glVertex2f(-2.9,-4.75);glVertex2f(-0.9,-4.75);  
glEnd();
```



Primitivas geométricas básicas

Patrones de Líneas

Adicional al cambio del ancho de líneas, se pueden crear líneas con un patrón punteado, rayado o guiones, denominado patrón.

Función `glLineStipple`

Especifica un patrón para los primitivos `GL_LINES`, `GL_LINE_LOOP` y `GL_LINE_STRIP`

`Void glLineStipple(Glintfactor, Glushortpattern)`

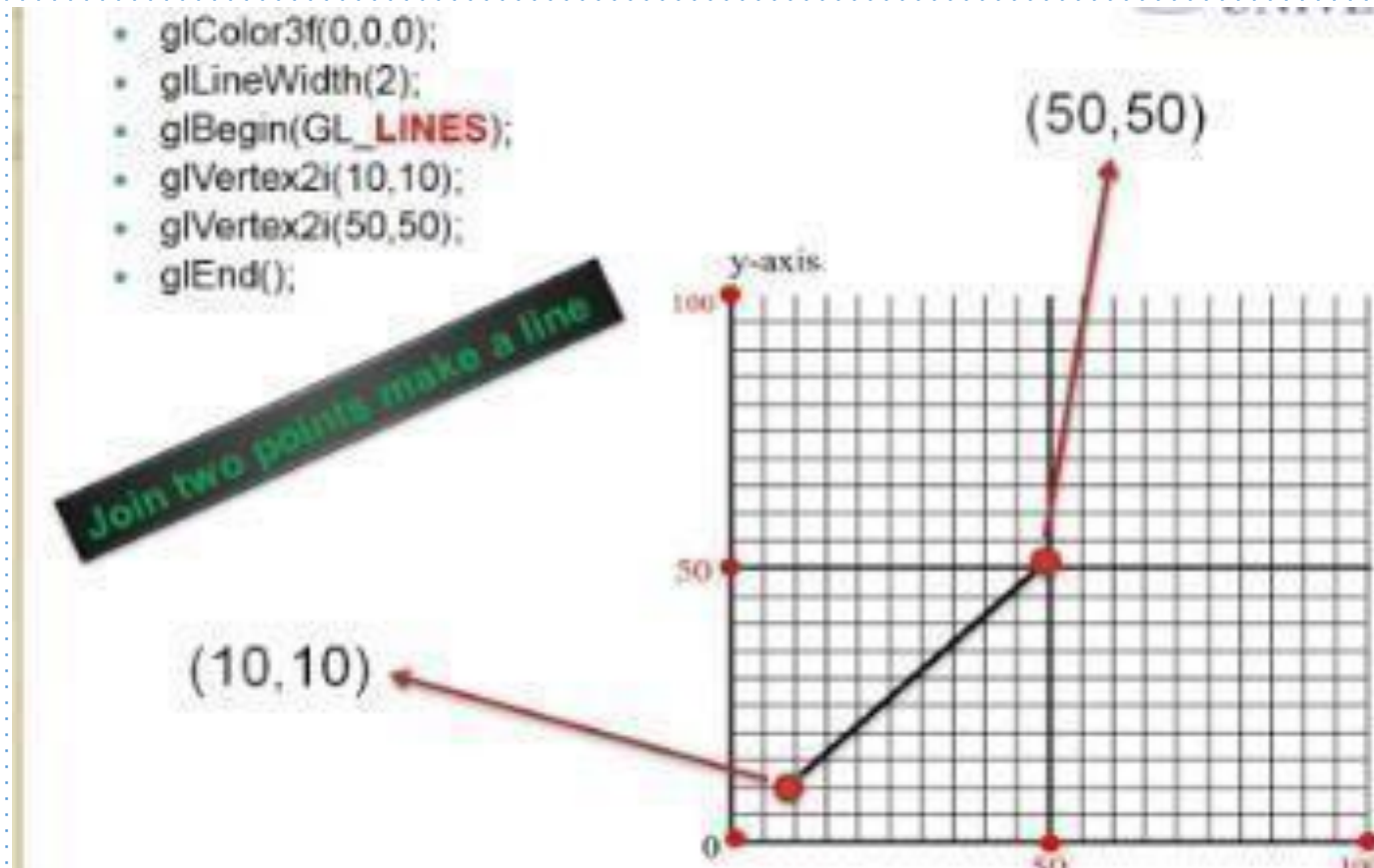
Primitivas geométricas básicas

Patrones de Líneas

En la tabla se muestran algunos ejemplos de patrones de líneas de la función **GLLineStipple**

PATTERN	FACTOR	
0x00FF	1	_____
0x00FF	2	_____
0x0C0F	1	— — — — —
0x0C0F	3	_____
0xAAAA	1	- - - - -
0xAAAA	2	- - - - -
0xAAAA	3	— — — — —
0xAAAA	4	_____

Ejemplos de Líneas



Habilitar o Desactivar características de OpenGL

Función glEnable(): activa o habilita una característica de OpenGL

Sintaxis:

Void glEnable(GlEnum características)

Habilitar o Desactivar características de OpenGL

Entre las características de la función glEnable() tenemos:

Estados	Descripción	Estados	Descripción
GL_BLEND	Fundido de color	GL_LINE_SMOOTH	Suavizado de línea
GL_CULL_FACE	Seleccionar polígono	GL_LINE_STIPPLE	Puntero de línea
GL_DITHER	Tramado	GL_POINT_SMOOTH	Suavizado de punto
GL_DEPTH_TEST	Prueba detallada	GL_POLYGON_SMOOTH	Suavizado de polígono
GL_FOG	Modo de niebla	GL_SCISSOR_TEST	Recorte habilitado
GL_LIGHTING	Iluminación	GL_STENCIL_TEST	Prueba de plantilla
GL_LIGHTx	Número de luz x	GL_TEXTURE_xD	Textura de xD (1,2,3)

Habilitar o Desactivar características de OpenGL

Función `glDisable()`: Inactiva o deshabilita una característica de estado de OpenGL

Sintaxis:

`Void glDisable (GLenum características)`

El cuadro anterior provee un listado de ciertos estados de OpenGL

Tipos de Datos en OpenGL

Sufijo	Tipo de datos	Tipo de Datos en C	Definición de tipo OpenGL
b	Entero 8 bits	signed char	GLbyte
s	Entero 16 bits	Short	GLshort
i	Entero 32 bits	Long	GLint, GLsizei
f	Real 32 bits	Float	GLfloat, GLclampf
d	Real 64 bits	Double	GLdouble, GLclampd
ub	Entero sin signo 8 bits	unsigned char	GLubyte, GLboolean
us	Entero sin signo 16 bits	unsigned short	GLushort
ui	Entero sin signo 32 bits	unsigned long	GLuint, GLenum, GLbitfield

Modos de Polígonos

Los polígonos son dibujos de elementos sólidos de relleno que se pueden crear no sólo como esquemas o como puntos trazando los vértices, la función que se encarga de esta tarea es el comando de OpenGL **glPolygonMode**.

Sintaxis: void glPolygonMode (GLenum face, GLenum mode)

El parámetro **face** especifica la cara del polígono que se verá afectado por el cambio del modo

El parámetro **mode** especifica el nuevo modo del dibujo

Modos de Polígonos

Función **glFrontFace** especifica el lado frontal o posterior del polígono.

Sintaxis: void glFrontFace (GLenum mode)

El parámetro **mode** define la orientación de los polígonos que miran al frente.

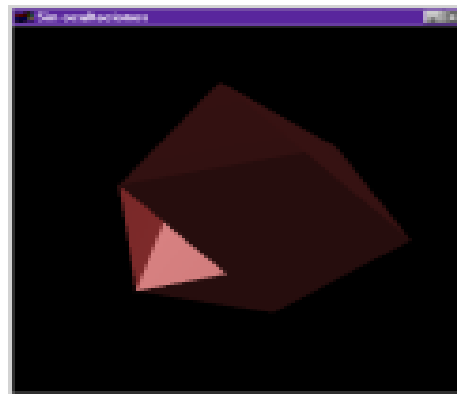
Si la orientación es en el sentido de las manecillas del reloj se usa **GL_CW**, pero si la orientación es en sentido contrario a las manecillas del reloj se usa **GL_CCW**

Modos de Polígonos

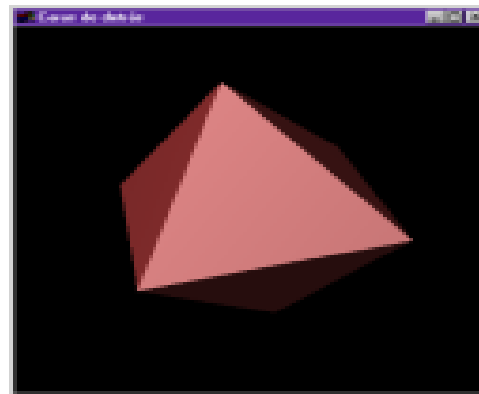
Función **glCullFace** especifica si se deben eliminar del dibujo los polígonos frontales o posteriores.

Su sintaxis: glCullFace (GLenum mode)

El parámetro **mode** define la cara del polígono que se va a eliminar; puede ser `GL_FRONT` o `GL_BACK`



Sin ocultaciones



Ocultando caras de detrás

Modos de Polígonos

Función **glEdgeFlag** define los bordes de polígonos como bordes de límites o borde de no límites.

Su sintaxis: glEdgeFlag (Glboolean flag)

El parámetro **flag** establece el indicador de borde para este valor:
true o false

Sistema de Coordenadas

En el caso de los objetos 2D, la ventana universal se selecciona con la función **gluOrtho2D**, y el puerto de vista se selecciona con la función **glViewport**.

Sintaxis:

void gluOrtho2D(Gldouble left, Gldouble right, Gldouble botón, Gldouble top)

- **left:** coordenada izquierda del volumen de recorte
- **right:** coordenada derecha del volumen de recorte
- **boton:** coordenada inferior del volumen de recorte
- **top:** coordenada superior del volumen de recorte

Sistema de Coordenadas

- La función OpenGL que permite establecer una ventana dentro de la cual se puede dibujar es la orden **glViewport**.

Sintaxis: void glViewport(GLint x, GLint y, GLsizei width, GLsizei height)

- **x**: número de píxeles desde la parte izquierda de la ventana para iniciar el puerto de vista.
- **y**: número de píxeles desde la parte inferior de la ventana para iniciar el puerto de vista.
- **Width, height**: Determina la anchura y altura del puerto de vista, respectivamente

OJO: Como OpenGL usa matrices para todas las transformaciones, **gluOrtho2D** debe estar precedida por dos funciones

1. **glMatrixMode(GL_PROJECTION)**
2. **glLoadIdentity()**

Sistemade Coordenadas

- Ejemplo de sistema de coordenadas

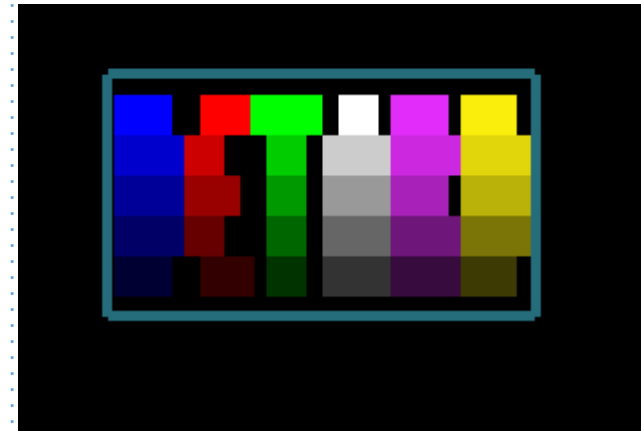
```
void proyeccion()  
{  
    glClearColor(0,0,0,0.0);  
    glMatrixMode(GL_PROJECTION);  
    gluOrtho2D(-45,15,-30,30);  
}
```



Sistemade Coordenadas

- Ejemplo de sistema de coordenadas
- ¿Qué pasa si cambiamos la coordenada inferior?

```
void proyeccion()  
{  
    glClearColor(0, 0, 0, 0.0);  
    glMatrixMode(GL_PROJECTION);  
    gluOrtho2D(-45, 100, -30, 30);  
}
```



Sistemade Coordenadas

- Ejemplo de sistema de coordenadas
- ¿Qué pasa si cambiamos la coordenada izquierda?

```
void proyeccion()  
{  
    glClearColor(0,0,0,0.0);  
    glMatrixMode(GL_PROJECTION);  
    gluOrtho2D(-15,15,-30,30);  
}
```



¡GRACIAS!

